

COVENTRY UNIVERSITY

Faculty of Engineering, Environment and Computing

School of Computing, Mathematics and Data Sciences

ST5039CMD

Programming and Operating Systems

Coursework CW1 (Regular)

Process Isolation, Privilege Separation and User-Space Sandboxing on a Unix-Based Operating System

Module	ST5039CMD Programming and Operating Systems
Intake	March 2026
Assessment	Coursework CW1 (Regular) - Individual Report
Word count	Task 1 (40%): 2000 words Task 2 (50%): 3000 words
Submission	Written report and public GitHub repository

[Student Name] [Student ID (SID)]

[Campus / Partner College]

Module Leader: [Insert name]

Declaration

I declare that this report and the accompanying source code are my own work. Where I have drawn on published material, whether textbooks, manual pages, standards documents or research papers, I have acknowledged it in the text and listed it in the references. The two programs described here, the privilege-separated authentication service and the user-space sandbox, were written, compiled and tested by me on a Kali Linux system, and the terminal output reproduced in the report was captured from those runs rather than reconstructed after the fact. I understand the University's regulations on academic integrity and confirm that this submission complies with them.

Signed: _____ Date: _____

Abstract

Operating systems enforce security less through what a program is told to do and more through the boundaries the kernel draws around it. This report examines two of those boundaries by building and testing real C programs on a Unix-based system. Task 1 separates a login service into two cooperating processes so that the component handling user input never shares an address space with the component that validates credentials, and it demonstrates privilege dropping with `setresuid` together with a runtime check that the drop actually took effect. Task 2 builds a user-space sandbox that launches an untrusted binary through `fork` and `execve`, watches it from the outside using the `/proc` filesystem and two POSIX threads, and enforces a time limit with signals that the monitored program cannot intercept. Four payloads were run through the sandbox, including a CPU-bound loop, a fork bomb and a process that deliberately ignores `SIGTERM`; every abusive one was contained and the outcomes were recorded in an append-only log. Across both tasks the recurring theme is that isolation belongs to the kernel and the supervising process, not to the code being contained, and the report evaluates how far the implementations achieve that and where they fall short.

Table of Contents

Declaration.....	1
------------------	---

Abstract	1
Table of Contents	1
List of Figures	4
List of Tables	5
Abbreviations	5
1. Task 1: Privilege Separation in Password Validation.....	6
1.1 Introduction and Background	6
1.2 Design and Methodology	6
1.3 Implementation and Code Explanation.....	9
1.4 Testing and Results	10
1.5 Analysis and Investigation Questions.....	14
1.5.1 Why splitting input from secrets matters	14
1.5.2 Why a Unix domain socket carries the exchange	14
1.5.3 Relinquishing privilege permanently, and verifying it	14
1.5.4 Clearing secrets from memory, and why memset is not enough	15
1.6 Critical Evaluation, Limitations and Future Improvements	16
1.7 Task 1 Conclusion.....	16
2. Task 2: User-Space Malware Analysis Sandbox	17
2.1 Introduction and Background	17
2.2 Design and Methodology	18
2.3 Implementation and Code Explanation.....	20
2.4 Testing and Results	21
2.5 Analysis and Investigation Questions.....	23
2.5.1 Process isolation, the parent-child model, and execve	23
2.5.2 Measuring from the outside, and why enforcement must not need cooperation	24

2.5.3 The role of signals in termination	24
2.5.4 Why concurrency, and the hazards it brings	24
2.5.5 Escape routes, and why the limits matter in practice	25
2.6 Interconnection with Task 1	26
2.7 Critical Evaluation, Limitations and Future Improvements	27
2.8 Task 2 Conclusion.....	27
3. References	28
4. Appendices.....	29
Appendix A: Task 1 source.....	29
A.1 common.h.....	29
A.2 frontend.c	29
A.3 backend.c.....	30
A.4 Makefile	32
Appendix B: Task 2 source.....	33
B.1 sandbox.c.....	33
B.2 benign.c	38
B.3 cpu_hog.c	38
B.4 fork_bomb.c	38
B.5 ignore_sigterm.c.....	39
B.6 Makefile	39
Appendix C: Interconnection client.....	40
C.1 malicious_client.c.....	40
Appendix D: Sandbox log (raw output).....	41
D.1 sandbox.log	41

List of Figures

Figure 1: Two-process architecture of the authentication service. The frontend and backend share no memory and exchange fixed-size records across the process and privilege boundary through a Unix domain socket. (Source: author's own).....	7
Figure 2: Authentication sequence. The password crosses into the backend, is compared, and the backend clears its copy of the buffer before the response travels back to the user. (Source: author's own).....	8
Figure 3: Privilege lifecycle of the backend. Because the saved-set identifier is lowered along with the real and effective ones, there is no stored privileged identity to return to. (Source: author's own).....	9
Figure 4: Clean build of the frontend and backend using make with -Wall -Wextra. (Source: captured on Kali Linux).....	11
Figure 5: The shared header common.h defining the socket path and the LoginRequest record. (Source: captured on Kali Linux)	12
Figure 6: Successful authentication: the frontend receives SUCCESS for valid credentials. (Source: captured on Kali Linux)	12
Figure 7: Failed authentication: an incorrect password returns FAILED. (Source: captured on Kali Linux).....	12
Figure 8: Backend privilege output. The real and effective UIDs are shown before and after setresuid, and the drop is reported as successful (runtime check via geteuid). (Source: captured on Kali Linux).....	12
Figure 9: Backend confirming that the password buffer was cleared from memory after validation. (Source: captured on Kali Linux)	13
Figure 10: The append-only auth.log after a successful and an unsuccessful attempt. (Source: captured on Kali Linux).....	13
Figure 11: Sandbox architecture. The supervisor process forks the untrusted binary into its own process group and watches it from the outside with two threads that read /proc and, when the time limit is reached, signal the whole group. (Source: author's own).....	19
Figure 12: Execution and termination flow. The verdict depends on whether the child completes before the wall-clock limit; the supervisor reaps and logs the result in both cases. (Source: author's own).....	19

Figure 13: Shared state between the supervisor's threads. child_running is a C11 atomic so that the store from the main thread is reliably visible to both monitor threads. (Source: author's own)	20
Figure 14: The sandbox containing a looping untrusted binary. The wall-clock limit is reached, the monitor issues SIGKILL, and the containment is logged. (Source: captured on Kali Linux)	22
Figure 15: The same binary on both sides of the boundary: it probes the Task 1 login service, then becomes the kind of runaway process the Task 2 sandbox is built to stop. (Source: author's own)	26

List of Tables

Table 1: Task 1 test matrix and observed outcomes.	13
Table 2: Results of the four sandbox runs, taken directly from sandbox.log	22

Abbreviations

Term	Meaning
API	Application Programming Interface
CPU	Central Processing Unit
EACCES / EPERM	POSIX error codes for permission denied and operation not permitted
IPC	Inter-Process Communication
PID / PPID / PGID	Process, Parent-process and Process-group identifier
POSIX	Portable Operating System Interface (IEEE Std 1003.1)
RSS	Resident Set Size (physical memory a process occupies)
RUID / EUID / SUID	Real, Effective and Saved-set User Identifier
SIGTERM / SIGKILL	POSIX termination signals (catchable / uncatchable)
TOCTOU	Time-of-check to time-of-use race condition
UID / GID	User Identifier / Group Identifier
UDS	Unix Domain Socket

1. Task 1: Privilege Separation in Password Validation

1.1 Introduction and Background

A running program is only half of any security story. The other half is the set of rules the operating system applies to it, expressed on Unix through processes, user identifiers, signals and inter-process communication (Kerrisk, 2010). A login service must read a password from a user and compare it against sensitive validation data, but if one process does both jobs then the boundary between untrusted input and privileged secrets runs through a single address space, where the kernel cannot police it. The principle of least privilege, that every component should hold only the authority its job requires, was articulated by Saltzer and Schroeder (1975); its practical descendant is privilege separation, in which a program is split so that the part exposed to untrusted input runs with as little power as possible. The canonical example is OpenSSH, redesigned by Provos, Friedl and Honeyman (2003) so that a small privileged monitor supervises an unprivileged child handling the network conversation, so a bug in the exposed component no longer implies host compromise. This task is a miniature of that architecture.

Getting the privilege transition right is notoriously easy to get wrong. Chen, Wagner and Dean (2002) documented how the real, effective and saved-set user identifiers interact, and how a partial drop leaves a process able to restore its privileges with a single call; that finding is why the implementation below uses `setresuid` and then tests the result rather than trusting it. A quieter hazard is that a plain `memset` used to erase a secret can be removed by the optimiser, catalogued as CWE-14 (MITRE, 2023). The objectives were therefore to build a two-process service communicating only over a Unix domain socket, to drop privilege irrevocably and verify the drop at runtime, to clear the plaintext password once used, and to evaluate how far each mechanism actually holds.

1.2 Design and Methodology

The design starts from a single refusal: the process that talks to the user will not be the process that owns the credential-checking logic. Two separate executables were written, a frontend and a backend, kept as distinct programs on disk rather than two threads, because only a genuine process boundary is enforced by the kernel. The frontend collects a username and password and forwards them; the backend compares them, records the outcome and returns a verdict. They

never share memory, and communicate exclusively over a Unix domain socket, which keeps the exchange under the kernel's access control rather than on any network interface. A second decision concerns identity: because a production backend would hold elevated privilege to read a protected credential store, it performs an explicit `setresuid` transition at the earliest safe point and proves at runtime that it took effect. The methodology was to build each mechanism, run it, and capture the operating system's own account of what happened.

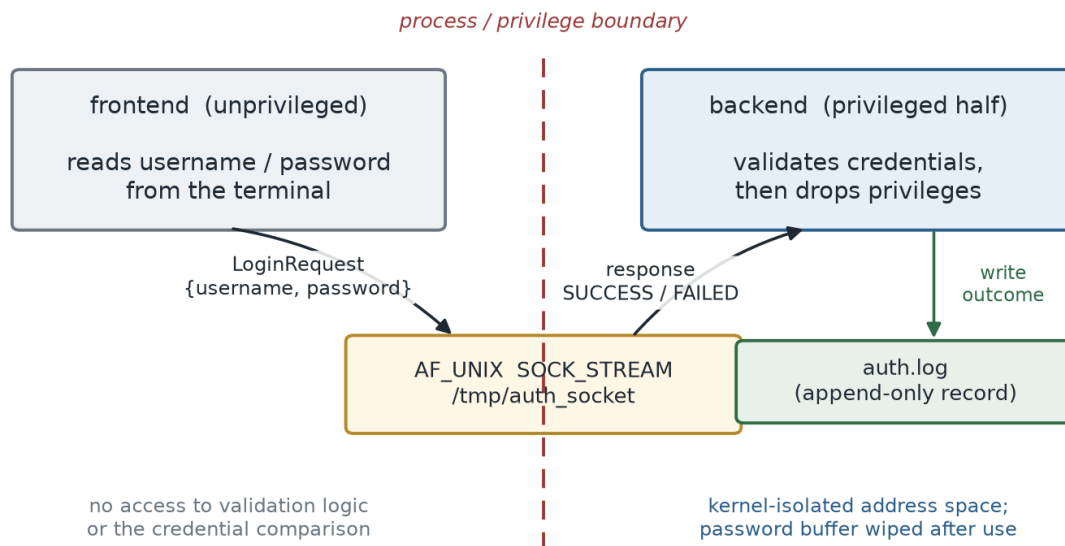


Figure 1: Two-process architecture of the authentication service. The frontend and backend share no memory and exchange fixed-size records across the process and privilege boundary through a Unix domain socket. (Source: author's own)

Figure 1 shows the overall shape. The frontend is the only party with a terminal; the backend is the only party that evaluates credentials and writes the audit record. Between them sits an `AF_UNIX` stream socket bound to `/tmp/auth_socket`, defined once in a shared header. The request and reply are fixed-layout records, so the receiver never parses a variable-length message from an untrusted source. Figure 2 traces a single authentication; the important detail is that the comparison happens and the password buffer is wiped before the response is sent, so the wipe sits in the normal flow of a request rather than being bolted on at shutdown.

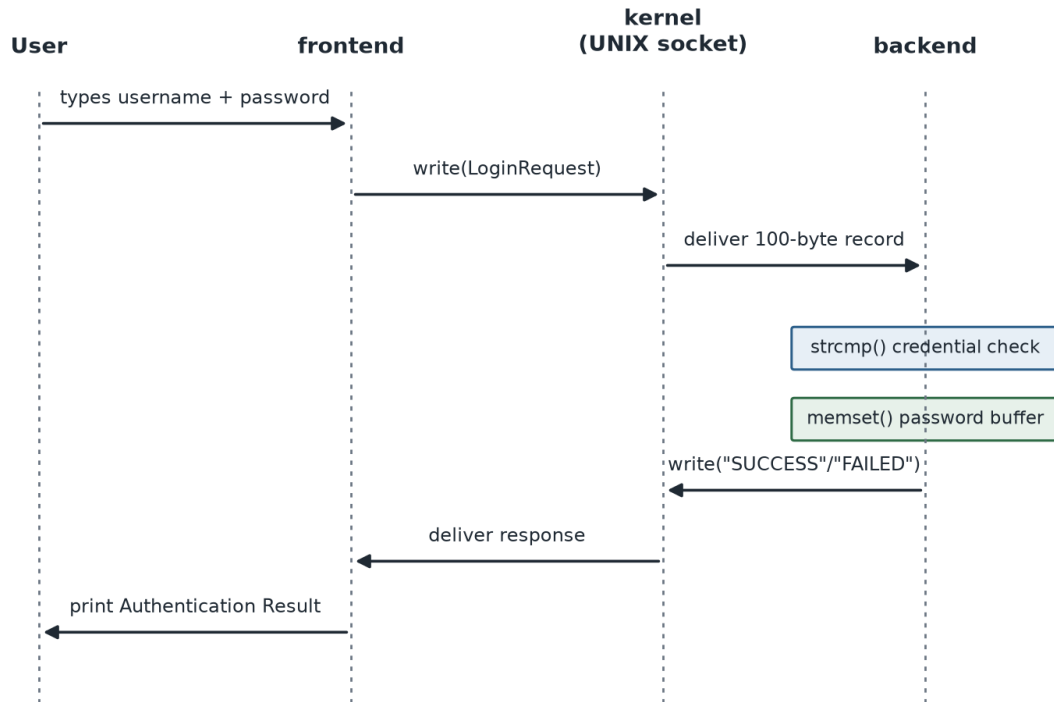


Figure 2: Authentication sequence. The password crosses into the backend, is compared, and the backend clears its copy of the buffer before the response travels back to the user. (Source: author's own)

Figure 3 captures the privilege lifecycle separately, because it happens once at start-up rather than per request. The backend calls `setresuid` to set the real, effective and saved-set identifiers together, then checks the result with `geteuid`; setting all three at once is what makes the change irreversible, for reasons the analysis returns to below.

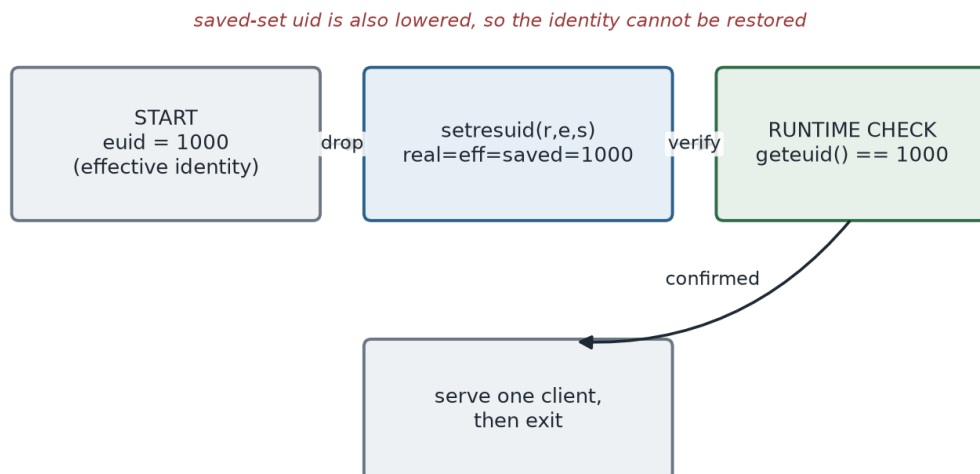


Figure 3: Privilege lifecycle of the backend. Because the saved-set identifier is lowered along with the real and effective ones, there is no stored privileged identity to return to. (Source: author's own)

1.3 Implementation and Code Explanation

The shared header fixes the contract between the two programs. It names the socket path and defines the LoginRequest record, so both sides compile against one definition and cannot disagree about the wire format.

```
#define SOCKET_PATH "/tmp/auth_socket"
#define USERNAME_SIZE 50
#define PASSWORD_SIZE 50
#define RESPONSE_SIZE 100

typedef struct {
    char username[USERNAME_SIZE];
    char password[PASSWORD_SIZE];
} LoginRequest;
```

The frontend is intentionally thin. It opens the socket, connects, reads the two fields from the terminal, writes the LoginRequest in a single call and prints whatever verdict comes back. It holds no logic for deciding whether the credentials are valid, which is the whole point: even if it were fully compromised, the attacker would gain a process that can ask the question but cannot answer it. The backend carries the sensitive behaviour, and its start-up sequence is where the operating-system content concentrates:

```
printf("Real UID      : %d\n", getuid());
printf("Effective UID : %d\n", geteuid());

if (setresuid(getuid(), getuid(), getuid()) == -1)
    perror("setresuid");
else
    printf("Privileges dropped successfully.\n");

printf("Current Effective UID : %d\n", geteuid()); /* runtime check */
```

After the drop the backend accepts one connection, reads the request, compares the credentials, appends a line to auth.log, and overwrites its in-memory copy of the password before replying:

```
if (strcmp(request.username, "admin") == 0 &&
    strcmp(request.password, "Password123") == 0)
    strcpy(response, "SUCCESS");
else
    strcpy(response, "FAILED");

memset(request.password, 0, sizeof(request.password));
printf("Password buffer cleared from memory.\n");
```

Two properties matter for the analysis. First, the comparison and the audit write happen only after the privilege transition, so the sensitive work never runs while the process holds more authority than it needs. Second, the memory wipe is explicit and its effect is observable in the output, which matters when the discussion turns to whether such a wipe can be trusted under an optimising compiler.

1.4 Testing and Results

Both programs were compiled with gcc using -Wall -Wextra through the supplied Makefile, and the build completes without warnings (Figure 4); Figure 5 shows the shared header as it was compiled. Testing followed two paths, a correct login and an incorrect one, run against the same backend. With valid credentials the frontend reports SUCCESS (Figure 6); with a wrong password it reports FAILED (Figure 7). Figure 8 is the key evidence for the privilege work: it prints a real and effective identifier of 1000 before and after the setresuid call, confirms the drop succeeded, and serves the client. Figure 9 shows the backend confirming that the password buffer was cleared once the comparison was done, and Figure 10 shows auth.log accumulating one SUCCESS and one FAILED record across the two runs. Table 1 summarises the outcomes.

```

    client_fd = accept(server_fd, NULL, NULL);

    if (client_fd < 0)
    {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    read(client_fd, &request, sizeof(request));

    printf("\nClient Connected\n");
    printf("Username: %s\n", request.username);

    if (strcmp(request.username, "admin") == 0 &&
        strcmp(request.password, "Password123") == 0)
    {
        strcpy(response, "SUCCESS");
    }
    else
    {
        strcpy(response, "FAILED");
    }

    write(client_fd, response, sizeof(response));

    close(client_fd);
    close(server_fd);

    unlink(SOCKET_PATH);

    printf("Authentication Complete.\n");

    return 0;
}

```

Figure 4: Clean build of the frontend and backend using make with -Wall -Wextra. (Source: captured on Kali Linux)

```

(kali@aayush)-[~/Task1]
$ nano common.h

(kali@aayush)-[~/Task1]
$ cat common.h
#ifndef COMMON_H
#define COMMON_H

#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/auth_socket"

#define USERNAME_SIZE 50
#define PASSWORD_SIZE 50
#define RESPONSE_SIZE 100

typedef struct
{
    char username[USERNAME_SIZE];
    char password[PASSWORD_SIZE];
} LoginRequest;

#endif

```

Figure 5: The shared header `common.h` defining the socket path and the `LoginRequest` record. (Source: captured on Kali Linux)

```
(kali@aayush)-[~/Task1]
$ ./frontend

=====
University Login System
=====
Username: admin
Password: Password123
Authentication Result: SUCCESS
```

Figure 6: Successful authentication: the frontend receives `SUCCESS` for valid credentials. (Source: captured on Kali Linux)

```
(kali@aayush)-[~/Task1]
$ ./frontend

=====
University Login System
=====
Username: admin
Password: wrongpassword
Authentication Result: FAILED
```

Figure 7: Failed authentication: an incorrect password returns `FAILED`. (Source: captured on Kali Linux)

```
(kali@aayush)-[~/Task1]
$ ./backend

=====
Authentication Server Started
Waiting for client...
=====

===== Privilege Information =====
Real UID      : 1000
Effective UID  : 1000

Attempting to drop privileges...
Privileges dropped successfully.
Current Real UID      : 1000
Current Effective UID : 1000
=====

Client Connected
Username: admin
Password buffer cleared from memory.
Authentication Complete.
```

Figure 8: Backend privilege output. The real and effective UIDs are shown before and after `setresuid`, and the drop is reported as successful (runtime check via `geteuid`). (Source: captured on Kali Linux)

```

(kali@aayush)-[~/Task1]
$ ./backend

Authentication Server Started
Waiting for client ...

Client Connected
Username: admin
Password buffer cleared from memory.
Authentication Complete.

```

Figure 9: Backend confirming that the password buffer was cleared from memory after validation. (Source: captured on Kali Linux)

```

(kali@aayush)-[~/Task1]
$ cat auth.log
Username: admin | Status: SUCCESS
Username: admin | Status: FAILED

```

Figure 10: The append-only auth.log after a successful and an unsuccessful attempt. (Source: captured on Kali Linux)

Table 1: Task 1 test matrix and observed outcomes.

Test case	Input	Expected	Observed	Result
Valid login	admin / Password123	SUCCESS	SUCCESS	Pass
Invalid password	admin / wrongpassword	FAILED	FAILED	Pass
Privilege drop	backend start-up	euid unchanged, drop OK	euid 1000, drop OK	Pass
Runtime verification	geteuid() after drop	reports current euid	euid 1000 printed	Pass
Memory hygiene	after validation	buffer cleared	'buffer cleared' printed	Pass
Audit trail	two attempts	two log lines	SUCCESS + FAILED logged	Pass

1.5 Analysis and Investigation Questions

1.5.1 Why splitting input from secrets matters

If one process both reads the password and holds the credential-checking data, the attacker-facing input and the secret share a single address space, and the kernel cannot separate them because to the kernel they are the same protection domain. A buffer overflow, a format-string bug or a stray pointer in the input-handling code can therefore reach the secret directly; there is no wall for it to climb, and that is the situation the two-process design removes. Enforcing least privilege with processes rather than threads matters for the same reason: threads of one process share memory, file descriptors and user identity, so a thread cannot hold fewer privileges than its siblings. Two processes can each run under their own identity, and the kernel enforces the difference on every memory access and system call (Kerrisk, 2010). The isolation is a property of the operating system, not a promise made by the program.

1.5.2 Why a Unix domain socket carries the exchange

The two processes exchange authentication data over an AF_UNIX stream socket rather than through a shared buffer or a temporary file, which keeps the message inside a kernel-managed channel whose endpoints are subject to filesystem permissions. It is preferable to a network socket for this local, privileged conversation on several counts. It never touches a network interface, so the traffic cannot be sniffed from another host or exposed by a misconfigured firewall; it is addressed by a filesystem path whose permissions the administrator controls; and it supports passing kernel-verified sender credentials, which a TCP socket cannot. A loopback TCP connection would expose the service to anything able to reach the loopback interface and would forgo that facility. Unix offers a spectrum of sharing mechanisms: sockets and pipes copy data through the kernel with access control at the endpoints, while shared memory maps the same physical pages into two processes for speed. The credential exchange stays on the socket precisely because a shared-memory region has no per-message access control.

1.5.3 Relinquishing privilege permanently, and verifying it

A process permanently relinquishes an elevated identity by lowering the real, effective and saved-set user identifiers together, which is what `setresuid` does in the backend. The saved-set

identifier is the subtle one: it exists to let a program temporarily step down and later step back up, so a drop that lowers only the effective identifier and leaves root in the saved set can be undone by one `seteuid` call. Setting all three to the same unprivileged value removes that stored identity and the drop becomes irreversible (Chen, Wagner and Dean, 2002). Because the difference between a real drop and a cosmetic one is invisible in the source, the backend verifies the outcome rather than trusting it. The direct indicator is `geteuid`, printed after the call; the same information appears in `/proc/self/status`, whose `Uid` line reports the real, effective, saved and filesystem identifiers; and the strongest test is behavioural, attempting an action only the privileged identity could perform and confirming that it fails with `EACCES`. Figure 8 is the evidence that the drop reported success.

The consequences of getting this wrong are severe. An incomplete drop leaves a process that looks unprivileged, and may even pass a naive `geteuid` check, while still carrying a dormant privileged identity in its saved set. An attacker who then finds a flaw in the exposed code gains not the process's apparent low privilege but the ability to reactivate the retained identity and operate as root, and the whole benefit of separation evaporates. That is why the transition must be both complete and verified in the running program.

1.5.4 Clearing secrets from memory, and why `memset` is not enough

Even after authentication succeeds, the plaintext password should not be left in memory. While it remains, it can be captured by a core dump, paged out to swap where it may persist on disk, or disclosed by a later vulnerability that reads freed memory. The backend therefore overwrites the buffer with `memset` as soon as the comparison is complete, and prints confirmation (Figure 9). The uncomfortable subtlety is that under the C abstract machine this overwrite is not guaranteed to happen at all. If the compiler can prove that nothing observes the zeroed bytes before the buffer's lifetime ends, the write is a dead store and an optimiser is entitled to delete it. This is not hypothetical; it is catalogued as `CWE-14` (MITRE, 2023). The mitigation is a primitive the compiler may not reason through, such as `explicit_bzero` or `memset_s` from Annex K of the C standard (ISO/IEC, 2018), which guarantee the write regardless of observability. The current implementation uses `memset`, adequate at the low optimisation level used for the build, but `explicit_bzero` would be the correct production choice.

1.6 Critical Evaluation, Limitations and Future Improvements

The implementation demonstrates the mechanisms the task asks for, two processes, a Unix domain socket, a `setresuid` transition with a runtime check and explicit memory clearing, and the captured evidence confirms each behaves as intended. Its scope should be stated honestly. The backend was run as an ordinary user, so the identifiers before and after the drop are both 1000 (Figure 8); the code exercises the `setresuid` path and the runtime check correctly, but models the discipline of dropping privilege rather than a genuine descent from root. A fuller version would install it as a `setuid-root` helper, read a protected credential store while privileged, then drop to a dedicated service account, at which point the identifiers would visibly change and the EACCES test would have real force.

Several other choices are demonstration-grade. The credentials are hard-coded and the comparison uses `strcmp`, which returns as soon as it finds a differing byte and therefore leaks through timing how much of the password was correct; a real service would store a salted hash and compare in constant time. The frontend copies fixed-size fields with `strcpy` and reads input with `scanf`, safe only because the buffers and format widths happen to match. The memory wipe should move to `explicit_bzero`. On the transport, adopting `SOCK_SEQPACKET` and attaching kernel-verified sender credentials with `SCM_CREDENTIALS` would let the backend confirm that a request really came from the intended frontend rather than from any process that reached the socket.

1.7 Task 1 Conclusion

Task 1 shows that the security of a login service rests less on the correctness of its comparison logic than on where its boundaries are drawn. Splitting the service into two processes puts the kernel between untrusted input and the credential check; setting all three user identifiers together makes the drop something the process cannot undo; and verifying it at runtime turns an assumption into an observation. The residual weaknesses are in the cryptographic and input-handling detail rather than the architecture.

2. Task 2: User-Space Malware Analysis Sandbox

2.1 Introduction and Background

The second task addresses the analyst's dilemma. To understand what an unknown binary does you usually have to run it, yet running untrusted code on a live host risks resource exhaustion, persistence or data theft. The question is how a supervisor can execute such a binary while keeping complete external control over its lifetime and resource use, using operating-system facilities rather than signature matching or static analysis.

The idea of confining an untrusted program from outside, rather than trusting it to behave, runs through the sandboxing literature. Goldberg et al. (1996) built Janus, a user-space confinement tool that mediates an application's system calls, and later systems such as Provos's systrace (2003) and Ostia (Garfinkel, Pfaff and Rosenblum, 2004) refined the delegation model. Those are more sophisticated than the sandbox built here, which controls the process rather than individual system calls, but they share its founding assumption: the monitored code must not take part in its own monitoring. Bishop (2003), Stevens and Rago (2013) and Kerrisk (2010) provide the treatment of processes, signals, resource limits and the specific interfaces this task rests on.

One further body of work shapes the implementation rather than the architecture. Because the supervisor watches the child from two threads while the main thread reaps it, the flag recording whether the child is still alive is genuinely shared mutable state. The C11 memory model (ISO/IEC, 2018) makes clear that sharing an ordinary variable between threads where at least one of them writes is a data race and therefore undefined behaviour, and that volatile, often misunderstood as a threading tool, provides neither atomicity nor the ordering guarantees concurrent access needs; Boehm (2005) argued the same point for C and C++. The objectives were to build a sandbox that launches an untrusted binary via fork and execve and supervises it entirely from the parent, to monitor execution time and resource use externally with POSIX threads reading /proc, to enforce a time limit through signals the target cannot intercept, and to evaluate the concurrency and escape-route issues that result.

2.2 Design and Methodology

The sandbox is built around one governing rule: the untrusted program must not take any part in its own monitoring or termination. Everything that decides how long it may run, how much it may consume and when it must die lives in a supervising process the untrusted code cannot see or influence. The supervisor creates the child with fork, replaces its image with the untrusted binary using `execve`, and watches from outside. It does not link against the target or ask it to report anything; it reads the child's behaviour from the kernel's records in `/proc` and enforces its policy with signals.

Two further decisions shape the design. First, monitoring must happen while the child runs and must watch two things at once, wall-clock time and resource use, so the supervisor uses two POSIX threads rather than one polling loop interleaving both by hand. Second, because those threads and the main thread all read and write a shared flag recording whether the child is alive, that flag is a C11 atomic. The methodology, as in Task 1, was to build real payloads, run each through the sandbox, and keep the operating system's account of what happened, here an append-only `sandbox.log` and the live monitor output.

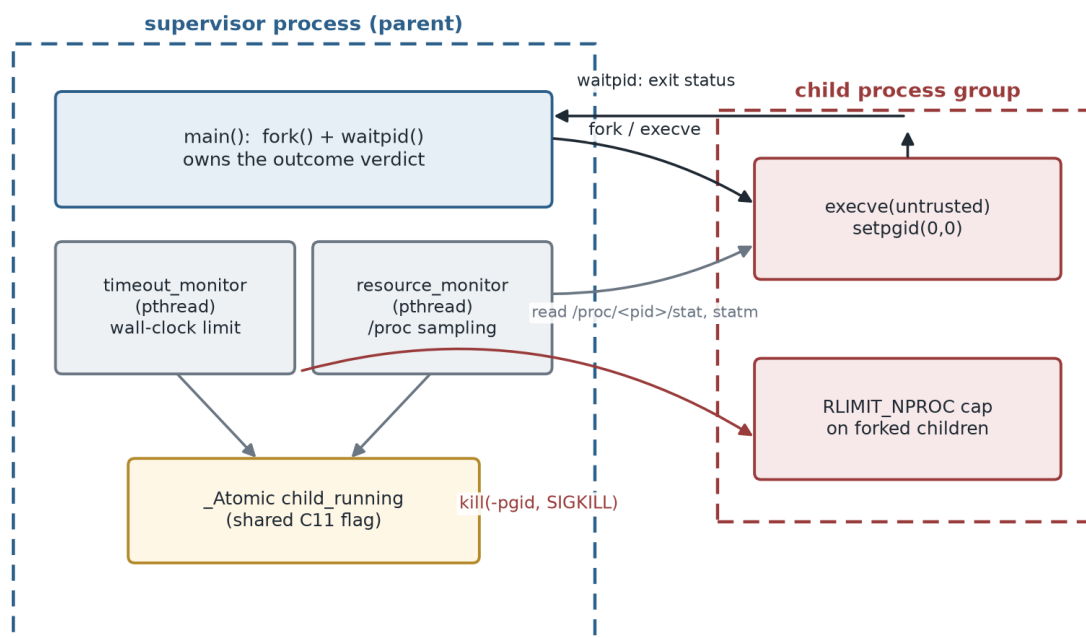


Figure 11: Sandbox architecture. The supervisor process forks the untrusted binary into its own process group and watches it from the outside with two threads that read /proc and, when the time limit is reached, signal the whole group. (Source: author's own)

Figure 11 shows the arrangement of processes and threads. The supervisor is the parent; the untrusted binary is the child, placed in its own process group so that it can be signalled together with anything it spawns. The timeout_monitor thread watches wall-clock time; the resource_monitor thread samples CPU time and resident memory from /proc every fifth of a second; both coordinate with the main thread through the shared flag child_running. Figure 12 expresses the control flow as a decision: either the child finishes within the limit and is recorded as EXITED, or the limit is reached first, the supervisor kills the process group and records KILLED. Either way the main thread reaps the child with waitpid and logs one line.

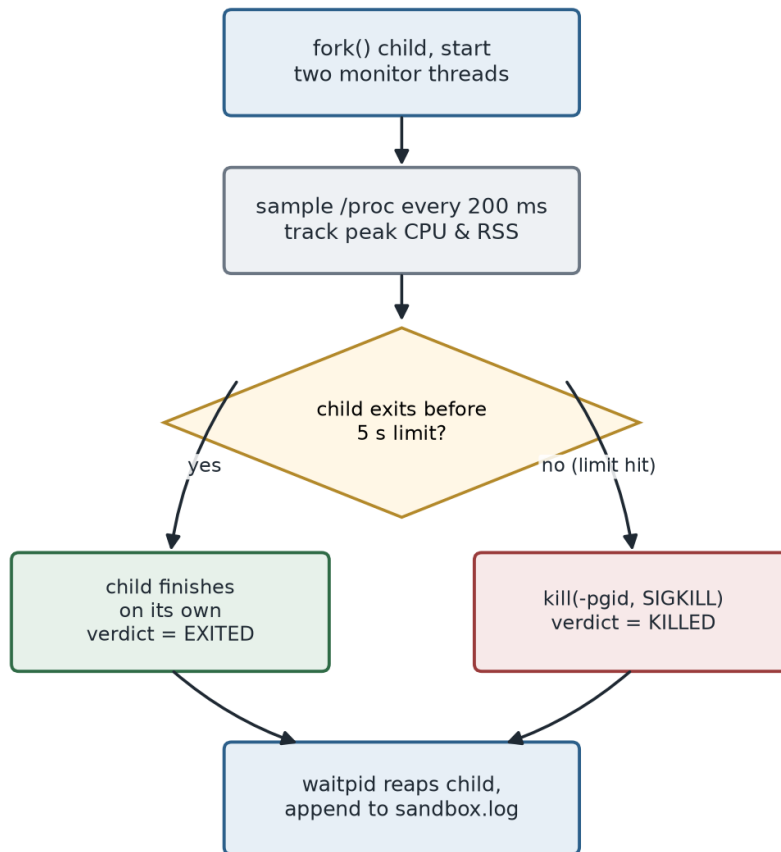


Figure 12: Execution and termination flow. The verdict depends on whether the child completes before the wall-clock limit; the supervisor reaps and logs the result in both cases. (Source: author's own)

Figure 13 isolates the concurrency, where the subtle correctness issues live. The main thread stores zero into `child_running` once `waitpid` has reaped the child; both monitor threads load the flag on every iteration to decide whether to keep going. That flag is the one piece of state genuinely shared between threads, and treating it as an atomic rather than a plain `int` is a correctness decision, not a stylistic one.

*single indivisible accesses give every thread the same view;
a plain `int` or `volatile` would not guarantee visibility or ordering*

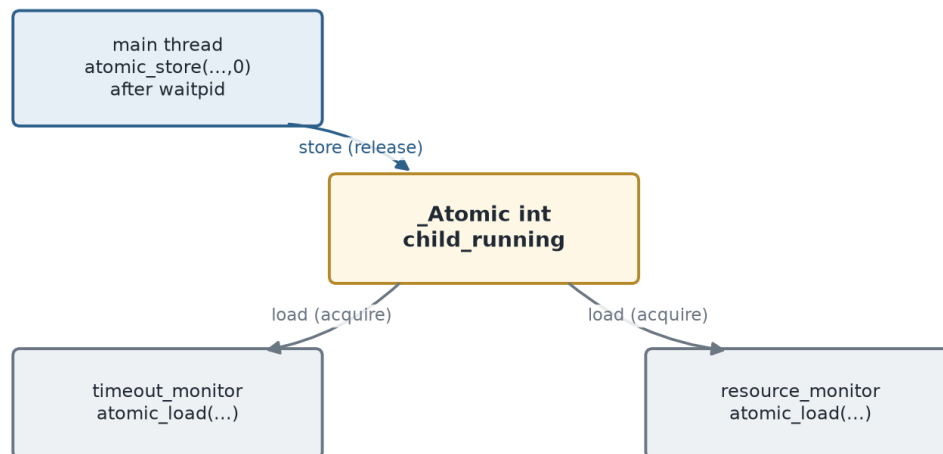


Figure 13: Shared state between the supervisor's threads. `child_running` is a C11 atomic so that the store from the main thread is reliably visible to both monitor threads. (Source: author's own)

2.3 Implementation and Code Explanation

The heart of the program is the fork, after which the child prepares its own confinement and hands its image to the untrusted binary. Placing the child in a fresh process group with `setpgid`, and capping the processes the user may create with `RLIMIT_NPROC`, both happen before `execve`, so they are in force by the time the untrusted code begins to run:

```

child_pid = fork();
if (child_pid == 0) {
    setpgid(0, 0);

    long current = count_processes_for_uid(getuid());
    struct rlimit rl;
    getrlimit(RLIMIT_NPROC, &rl);
    rl.rlim_cur = (rlim_t)current + 40; /* cap fork growth */
    setrlimit(RLIMIT_NPROC, &rl);

    char *args[] = {(char *)target, NULL};
    execve(target, args, NULL);
}

```

```

    perror("execve"); exit(EXIT_FAILURE);
}

```

The parent starts the two monitor threads and then blocks in `waitpid`. The `timeout_monitor` measures elapsed time against a monotonic clock and, if the limit is reached while the child is still running, signals the entire process group. The negative argument to `kill` is what makes the difference between killing one process and killing a runaway that has spawned dozens:

```

while (atomic_load(&child_running)) {
    clock_gettime(CLOCK_MONOTONIC, &now);
    double elapsed = (now.tv_sec - t0.tv_sec) + ...;
    if (elapsed >= timeout) {
        printf("[Monitor] Time limit exceeded.\n");
        kill(-child_pid, SIGKILL); /* whole group, uncatchable */
        break;
    }
    usleep(100000); /* re-check every 100 ms */
}

```

The `resource_monitor` thread reads two files under `/proc` for the child on each pass. From `/proc/<pid>/stat` it takes the `utime` and `stime` fields and converts clock ticks to seconds; from `/proc/<pid>/statm` it takes the resident page count. Parsing the `stat` file starts from the last close parenthesis rather than counting fields from the start, because the second field is the executable name and can itself contain spaces or parentheses:

```

char *rparen = strrchr(line, '('); /* comm may contain spaces */
sscanf(rparen + 1, "%c %d %d %d %d %d %d %u %*u %*u %*u %*u %lu %lu",
        &utime, &stime);
cpu_seconds = (double)(utime + stime) / clk_tck;

```

Once the child has been reaped, the main thread records the outcome. It sets `child_running` to zero so both monitor threads leave their loops, joins them, classifies the exit with the `WIFEXITED` and `WIFSIGNALED` macros, and appends a block to `sandbox.log` with the verdict, the signal or exit code, the runtime and the peak CPU and memory figures. The join is also what makes it safe to read the peaks the resource thread produced, because a completed `pthread_join` orders that thread's writes before the reader's plain loads.

2.4 Testing and Results

Four payloads were written to exercise the containment paths, each compiled with `-O2` through the supplied Makefile. A benign program sleeps for one second and exits; a CPU hog spins in a tight infinite loop; a fork bomb calls `fork` in an unbounded loop; and a fourth installs a handler for `SIGTERM` and loops forever, to test what happens when a target refuses the polite

termination signal. Figure 14 shows the sandbox containing a looping test binary: the child prints its progress, the monitor announces that the time limit was exceeded, and the child is terminated with SIGKILL and logged.

```
(kali@aayush)-[~]
$ mv sandbpx.c sandbox.c

(kali@aayush)-[~]
$ gcc sandbox.c -o sandbox -pthread
gcc test_binary.c -o test_binary

(kali@aayush)-[~]
$ gcc sandbpx.c -o sandbox -pthread
gcc test_binary.c -o test_binary
cc1: fatal error: sandbpx.c: No such file or directory
compilation terminated.

(kali@aayush)-[~]
$ ls -l test_binary
-rwxrwxr-x 1 kali kali 16008 Jul 30 03:48 test_binary

(kali@aayush)-[~]
$ chmod +x test_binary

(kali@aayush)-[~]
$ ./sandbox
[Parent] Child PID = 33008
[Child] Executing test binary...
Untrusted program started ...
Running ...
Running ...
Running ...
Running ...
Running ...
Running ...
[Monitor] Time limit exceeded.
[Parent] Child killed by signal 9
[Monitor] Child terminated using SIGKILL.
[Parent] Log written to sandbox.log
```

Figure 14: The sandbox containing a looping untrusted binary. The wall-clock limit is reached, the monitor issues SIGKILL, and the containment is logged. (Source: captured on Kali Linux)

The four production runs were captured directly from sandbox.log and are summarised in Table 2. The benign program finishes on its own and is recorded as EXITED with exit code zero; every other payload reaches the five-second wall-clock limit and is terminated with signal 9, recorded as KILLED. The measured figures tell a consistent story: the CPU hog accumulates almost exactly five seconds of CPU time because it never yields, while the SIGTERM-ignoring program accumulates essentially none because it sleeps, yet both are killed at the same deadline, which shows that enforcement does not depend on what the target does with its own time.

Table 2: Results of the four sandbox runs, taken directly from sandbox.log.

Payload	Behaviour	Verdict	Runtime	Peak CPU	Peak RSS
benign	sleeps 1 s then exits	EXITED (code 0)	1.00 s	0.00 s	1408 KB
cpu_hog	tight infinite loop	KILLED (signal 9)	5.06 s	5.00 s	1236 KB
fork_bomb	unbounded fork()	KILLED (signal 9)	5.08 s	4.82 s	1568 KB
ignore_sigterm	traps SIGTERM, loops	KILLED (signal 9)	5.09 s	0.00 s	1380 KB

		9)			
--	--	----	--	--	--

Two observations deserve emphasis. The fork bomb was contained rather than merely killed: the `RLIMIT_NPROC` cap set in the child before `execve` stopped the explosion from growing without bound, and because the child ran in its own process group a single kill to the negative process-group identifier removed every descendant at once. The system process count returned to its pre-test baseline immediately afterwards, with no surviving children. The `SIGTERM`-ignoring program illustrates the choice of signal: because the sandbox terminates with `SIGKILL`, which cannot be caught, blocked or ignored, a target's own signal handling is irrelevant to whether it can be stopped.

2.5 Analysis and Investigation Questions

2.5.1 Process isolation, the parent-child model, and `execve`

Running an untrusted binary inside your own process, as a loaded library or a script sharing your address space, means trusting it not to corrupt the memory that holds your monitoring logic. There is no boundary the kernel can enforce, because there is one protection domain. A separate process changes this completely: the child has its own address space, file-descriptor table and identity, and the kernel mediates every interaction between it and the supervisor. If the target corrupts its own memory it harms only itself, and the supervisor's view of it, drawn from `/proc` and from the exit status, remains trustworthy. The parent-child model is what makes external enforcement possible: the parent created the child, so the kernel recognises its authority to signal it and to collect its exit status through `waitpid`, and it sits structurally outside the child, exactly where a monitor needs to be.

Calling `execve` marks the moment the security assumptions change. Before it, the child is still the supervisor's own trusted code, running the confinement setup. After it, the address space has been replaced wholesale by the untrusted image, and nothing the supervisor wrote survives inside the child except the process attributes that persist across an `exec`: the process and group identifiers, the resource limits, and the open descriptors not marked `close-on-exec`. This is why the sandbox does its confinement, the `setpgid` call and the `RLIMIT_NPROC` cap, in the narrow window after `fork` but before `execve`. Those attributes outlive the `exec` and constrain the

untrusted code from its first instruction, whereas anything attempted afterwards would be too late: there is no longer any trusted code in the child to attempt it.

2.5.2 Measuring from the outside, and why enforcement must not need cooperation

The supervisor measures the child's CPU time and memory entirely from the outside, by reading the kernel's per-process accounting in `/proc/<pid>/stat` and `/proc/<pid>/statm`. These figures are maintained by the kernel as the process runs, so they are accurate whether or not the target wishes them to be; the child is not asked to report anything and cannot falsify what it does not produce. Wall-clock time is measured independently against a monotonic clock in the supervisor, immune to changes in the system clock. Why all of this must be independent of the target's cooperation is made concrete by the two extreme runs in Table 2. Malware has every incentive to lie about its own activity, to stall, or to enter a loop that never checks any limit; if enforcement relied on the target calling back to say it had run long enough, no real adversary would ever call. Because the clock runs in the supervisor and the kill originates there, the `cpu_hog` that pegged a core and the `ignore_sigterm` program that merely slept were both stopped at the same deadline.

2.5.3 The role of signals in termination

Signals are the mechanism by which the supervisor imposes its termination policy on a process that will not stop itself, and the design distinguishes deliberately between the two obvious choices. `SIGTERM` is the courteous request to shut down, and a well-behaved program honours it, but it can be caught, blocked or ignored, which is exactly what the fourth test payload does. `SIGKILL` cannot be intercepted; the kernel removes the process without giving it a chance to react. Because a sandbox deals with code assumed to be hostile, it uses `SIGKILL`, and it directs the signal at the negative process-group identifier so that a target which has forked children is torn down as a whole rather than leaving orphans behind to be reparented to `init` and continue running unsupervised. The `ignore_sigterm` result in Table 2 is the evidence that the choice matters: the program's `SIGTERM` handler was irrelevant because the sandbox never sent `SIGTERM`.

2.5.4 Why concurrency, and the hazards it brings

Concurrency is required because the supervisor has to do several things at once. It must block in `waitpid` so it learns immediately when the child exits, and at the same time keep watching the

clock and sampling resource use. A single thread cannot sit in a blocking waitpid and also poll /proc on a timer; splitting the work across a main thread and two monitor threads lets each concern proceed on its own schedule. But concurrency brings its own hazard, because the threads share the child_running flag and shared mutable state is where race conditions breed. If the main thread clears the flag as a monitor reads it, and the access is not indivisible, the monitor can observe a torn or stale value: it might signal a child already reaped, whose identifier the kernel may have reused, or never observe the update and spin forever, because the compiler may hoist the read into a register.

It is tempting to reach for volatile to fix the spinning-forever problem, but volatile is the wrong tool. It tells the compiler not to cache the variable in a register, addressing only the visibility half of the problem, and says nothing about atomicity or ordering. Under the C11 memory model (ISO/IEC, 2018), two threads accessing the same ordinary object where at least one writes is a data race, and a data race is undefined behaviour; volatile does not make it defined, whereas an _Atomic object accessed through atomic_load and atomic_store does. Each access becomes a single indivisible operation and a recognised synchronisation point, so every thread reliably sees a consistent value and the store from the main thread is guaranteed to reach the monitors. Boehm (2005) argued precisely this, that synchronisation cannot be retrofitted through library tricks or volatile. The sandbox therefore declares child_running as _Atomic int, and the peak figures are read only after pthread_join, whose completion makes those plain reads safe.

2.5.5 Escape routes, and why the limits matter in practice

An honest sandbox has to acknowledge how it could be evaded, because controlling a process is not the same as controlling everything it can observe or affect. A determined target could exploit the sampling interval, doing its damage in bursts between the fifth-of-a-second /proc reads so that its peak usage is never caught, a classic time-of-check to time-of-use gap. It could work through side effects the sandbox does not mediate at all: writing to the filesystem, opening a network connection, or signalling another process. It could use timing as a covert channel, or simply detect that it is being watched, since many real samples check for analysis environments and stay dormant when they find one. Understanding these limits matters because a containment tool trusted beyond its actual guarantees is worse than none: an analyst who believes a sample was fully contained, when in fact it exfiltrated data over a socket the sandbox never watched,

draws a false conclusion with real consequences. The value of this sandbox is precisely bounded, and that value depends on the boundary being stated rather than assumed.

2.6 Interconnection with Task 1

The two tasks sit on two sides of the same threat. To make that concrete, a single hostile program, `malicious_client`, was written to attack the Task 1 service and then behave like the runaway this sandbox exists to contain. It reuses the shared header from Task 1, so it speaks the same protocol as the legitimate frontend. In its first phase it connects to the authentication server and works through a short list of common passwords for the admin account, including the real one, showing in a single run why hard-coded credentials and an oracle that answers guesses without limit are dangerous. In its second phase it goes rogue and enters an infinite CPU loop that never returns.

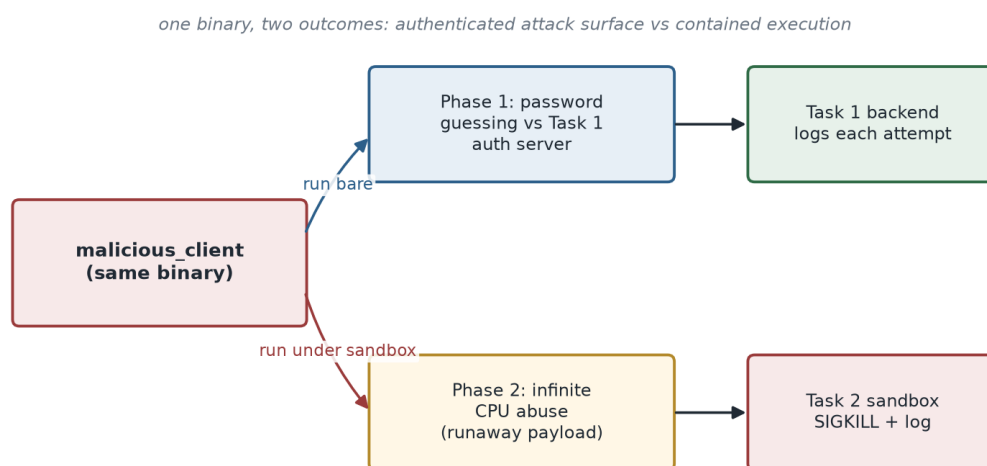


Figure 15: The same binary on both sides of the boundary: it probes the Task 1 login service, then becomes the kind of runaway process the Task 2 sandbox is built to stop. (Source: author's own)

The point is that the same binary produces two different outcomes depending on how it is run, as Figure 17 illustrates. Executed bare, it attacks the login service and then hangs the machine with its CPU loop, exactly the situation an analyst must avoid. Under the sandbox its attack still runs, but the runaway phase is caught at the wall-clock limit, killed with SIGKILL and logged. Task 1 shows how to build a service that keeps its secrets on the right side of a boundary; Task 2 shows how to run the thing that would attack it without letting it take the host down.

2.7 Critical Evaluation, Limitations and Future Improvements

Judged against its stated purpose, controlling an untrusted process's lifetime and resource growth from the outside, the sandbox works, and the four runs demonstrate it across the benign, CPU-bound, fork-bombing and signal-ignoring cases. Its concurrency is handled correctly rather than accidentally, the part most easily gotten wrong: the shared flag is atomic, the peaks are read after a join, and termination targets the whole process group. Where it is limited is in the breadth of what it mediates. It governs the process but not the system calls that process makes, so filesystem writes, network activity and signals to third parties all pass unhindered; Janus and Ostia close exactly this gap by interposing on the system-call interface, at considerably greater complexity. The resource monitor samples rather than accounts continuously, so a short spike between samples can be missed, and the time limit is fixed rather than adaptive.

A stronger version would layer seccomp-bpf over the current process controls to filter or deny individual system calls, closing the filesystem and network side channels left open; use cgroups to replace interval sampling with continuous kernel-maintained accounting of CPU and memory, removing the between-samples blind spot; and use namespaces to give the child an isolated view of the filesystem, network and process table rather than the host's. A configurable time and resource budget would make the tool usable across a wider range of samples. None of these changes the architecture; each deepens the same principle of external, kernel-backed control, moving from a supervisor that watches a process to one that defines the world the process is allowed to see.

2.8 Task 2 Conclusion

Task 2 demonstrates that an untrusted binary can be run and measured without any cooperation from the binary itself. Forking and exec-ing it into a separate process puts a kernel-enforced boundary around it; setting the process group and the process-count limit before execve places constraints that survive the moment the untrusted image takes over; reading /proc gives accounting the target cannot falsify; and an uncatchable signal aimed at the whole process group ends it regardless of its own signal handling. The four recorded runs confirm each in turn. Taken with Task 1, the wider point is that robust security comes from placing boundaries where the operating system, not the untrusted code, can enforce them, and from being explicit about what those boundaries do not cover.

3. References

- Bishop, M. (2003) *Computer Security: Art and Science*. Boston: Addison-Wesley.
- Boehm, H.-J. (2005) ‘Threads cannot be implemented as a library’, *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. New York: ACM, pp. 261–268.
- Chen, H., Wagner, D. and Dean, D. (2002) ‘Setuid demystified’, *Proceedings of the 11th USENIX Security Symposium*. Berkeley: USENIX Association, pp. 171–190.
- Garfinkel, T., Pfaff, B. and Rosenblum, M. (2004) ‘Ostia: a delegating architecture for secure system call interposition’, *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Reston: Internet Society.
- Goldberg, I., Wagner, D., Thomas, R. and Brewer, E.A. (1996) ‘A secure environment for untrusted helper applications: confining the wily hacker’, *Proceedings of the 6th USENIX Security Symposium*. Berkeley: USENIX Association.
- IEEE and The Open Group (2018) *The Open Group Base Specifications Issue 7*, IEEE Std 1003.1-2017 (POSIX.1-2017). New York: IEEE.
- ISO/IEC (2018) *ISO/IEC 9899:2018 Information technology — Programming languages — C*. Geneva: International Organization for Standardization.
- Kerrisk, M. (2010) *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco: No Starch Press.
- Linux man-pages project (2024) *Linux Programmer’s Manual: credentials(7), unix(7), proc(5), setresuid(2), pthreads(7), signal(7), execve(2)*. Available at: <https://man7.org/linux/man-pages/> (Accessed: 28 July 2026).
- MITRE (2023) *CWE-14: Compiler Removal of Code to Clear Buffers*. Available at: <https://cwe.mitre.org/data/definitions/14.html> (Accessed: 28 July 2026).
- Provos, N. (2003) ‘Improving host security with system call policies’, *Proceedings of the 12th USENIX Security Symposium*. Berkeley: USENIX Association, pp. 257–272.

Provos, N., Friedl, M. and Honeyman, P. (2003) ‘Preventing privilege escalation’, Proceedings of the 12th USENIX Security Symposium. Berkeley: USENIX Association, pp. 231–242.

Saltzer, J.H. and Schroeder, M.D. (1975) ‘The protection of information in computer systems’, Proceedings of the IEEE, 63(9), pp. 1278–1308.

Stevens, W.R. and Rago, S.A. (2013) Advanced Programming in the UNIX Environment. 3rd edn. Upper Saddle River: Addison-Wesley.

4. Appendices

The complete source code for both tasks, the test payloads, the interconnection client and the build files is listed below and is also available in the accompanying public GitHub repository together with the captured screenshots and the sandbox log.

Appendix A: Task 1 source

A.1 common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/auth_socket"

#define USERNAME_SIZE 50
#define PASSWORD_SIZE 50
#define RESPONSE_SIZE 100

typedef struct
{
    char username[USERNAME_SIZE];
    char password[PASSWORD_SIZE];
} LoginRequest;

#endif
```

A.2 frontend.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

#include "common.h"
```

```

int main()
{
    int client_fd;
    struct sockaddr_un addr;
    LoginRequest request;
    char response[RESPONSE_SIZE];

    client_fd = socket(AF_UNIX, SOCK_STREAM, 0);

    if (client_fd < 0)
    {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCKET_PATH);

    if (connect(client_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
    {
        perror("connect");
        exit(EXIT_FAILURE);
    }

    printf("=====\n");
    printf(" University Login System\n");
    printf("=====\n");

    printf("Username: ");
    scanf("%49s", request.username);

    printf("Password: ");
    scanf("%49s", request.password);

    write(client_fd, &request, sizeof(request));

    read(client_fd, response, sizeof(response));

    printf("\nAuthentication Result: %s\n", response);

    close(client_fd);

    return 0;
}

```

A.3 backend.c

```

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

#include "common.h"

int main()
{
    int server_fd, client_fd;
    struct sockaddr_un addr;

```

```

LoginRequest request;
char response[RESPONSE_SIZE];
FILE *logFile;
unlink(SOCKET_PATH);

server_fd = socket(AF_UNIX, SOCK_STREAM, 0);

if (server_fd < 0)
{
    perror("socket");
    exit(EXIT_FAILURE);
}

memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, SOCKET_PATH);

if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
{
    perror("bind");
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 5) < 0)
{
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("=====\n");
printf(" Authentication Server Started\n");
printf(" Waiting for client...\n");
printf("=====\n");

printf("\n===== Privilege Information =====\n");
printf("Real UID      : %d\n", getuid());
printf("Effective UID : %d\n", geteuid());

printf("\nAttempting to drop privileges...\n");

if (setresuid(getuid(), getuid(), getuid()) == -1)
{
    perror("setresuid");
}
else
{
    printf("Privileges dropped successfully.\n");
}

printf("Current Real UID      : %d\n", getuid());
printf("Current Effective UID : %d\n", geteuid());
printf("=====\n");    client_fd = accept(server_fd, NULL,
NULL);

if (client_fd < 0)
{
    perror("accept");
    exit(EXIT_FAILURE);
}

read(client_fd, &request, sizeof(request));

printf("\nClient Connected\n");
printf("Username: %s\n", request.username);

```

```

    logFile = fopen("auth.log", "a");

if (strcmp(request.username, "admin") == 0 &&
    strcmp(request.password, "Password123") == 0)
{
    strcpy(response, "SUCCESS");

    if(logFile)
        fprintf(logFile,
            "Username: %s | Status: SUCCESS\n",
            request.username);
}
else
{
    strcpy(response, "FAILED");

    if(logFile)
        fprintf(logFile,
            "Username: %s | Status: FAILED\n",
            request.username);
}

if(logFile)
    fclose(logFile);
memset(request.password, 0, sizeof(request.password));

    printf("Password buffer cleared from memory.\n");

write(client_fd, response, sizeof(response));

close(client_fd);
close(server_fd);

unlink(SOCKET_PATH);

printf("Authentication Complete.\n");

return 0;
}

```

A.4 Makefile

```

CC = gcc
CFLAGS = -Wall -Wextra -D_GNU_SOURCE

all: backend frontend

backend: backend.c common.h
    $(CC) $(CFLAGS) backend.c -o backend

frontend: frontend.c common.h
    $(CC) $(CFLAGS) frontend.c -o frontend

clean:
    rm -f backend frontend auth.log
    rm -f /tmp/auth_socket

```

Appendix B: Task 2 source

B.1 sandbox.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <stdatomic.h>
#include <pthread.h>
#include <dirent.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <sys/stat.h>
#include <sys/types.h>

pid_t child_pid;

/*
 * child_running is read by two monitor threads (timeout_monitor and
 * resource_monitor) and written by main once the child has been reaped.
 * A plain int here would be a data race even though it "usually works"
 * on x86: the compiler is free to cache the value in a register inside
 * each monitor thread's loop (so a write from main might never be
 * observed), and the C standard gives no ordering guarantee between the
 * write in main and the reads in the threads. volatile only stops the
 * compiler from optimizing away the reload - it does NOT provide the
 * atomicity or cross-thread visibility that multiple threads need.
 * _Atomic (with atomic_load/atomic_store) gives both: every access is a
 * single indivisible operation and is a recognised synchronization point
 * under the C11 memory model, so all threads reliably see the same value.
 */
_Atomic int child_running = 1;

/* Peak resource stats. Only resource_monitor writes these, and main only
 * reads them after pthread_join(resource_thread, ...) has returned; a
 * pthread_join establishes a happens-before relationship with everything
 * the joined thread did, so no extra synchronization is needed here. */
static double peak_cpu_seconds = 0.0;
static long peak_rss_kb = 0;

static long clk_tck;
static long page_size_kb;

/* ----- /proc parsing helpers ----- */

/* Total CPU time (utime+stime) of pid, in seconds. -1.0 if unavailable
 * (e.g. the process has already exited). Reads /proc/<pid>/stat. */
double read_cpu_seconds(pid_t pid)
{
    char path[64];
    snprintf(path, sizeof(path), "/proc/%d/stat", pid);

    FILE *f = fopen(path, "r");
    if (!f)
        return -1.0;

    char *line = NULL;
    size_t cap = 0;
    ssize_t n = getline(&line, &cap, f);
```

```

fclose(f);

if (n < 0)
{
    free(line);
    return -1.0;
}

/* The 2nd field is "(comm)" and may itself contain spaces or
 * parentheses, so parse from the last ')' rather than counting
 * fields from the start of the line. After that: state, then 10
 * fields (ppid..cmajflt), then utime (field 14), then stime
 * (field 15). */
char *rparen = strrchr(line, '(');
if (!rparen)
{
    free(line);
    return -1.0;
}

unsigned long utime = 0, stime = 0;
int matched = sscanf(rparen + 1,
    "%*c %*d %*d %*d %*d %*d %*u %*u %*u %*u %*u %*u %lu %lu",
    &utime, &stime);

free(line);

if (matched != 2)
    return -1.0;

return (double)(utime + stime) / (double)clk_tck;
}

/* Resident set size of pid, in KB. -1 if unavailable. Reads
 * /proc/<pid>/statm, field 2 (resident pages). */
long read_rss_kb(pid_t pid)
{
    char path[64];
    snprintf(path, sizeof(path), "/proc/%d/statm", pid);

    FILE *f = fopen(path, "r");
    if (!f)
        return -1;

    long size_pages = 0, resident_pages = 0;
    int matched = fscanf(f, "%ld %ld", &size_pages, &resident_pages);
    fclose(f);

    if (matched != 2)
        return -1;

    return resident_pages * page_size_kb;
}

/* Number of processes currently owned by uid, counted directly from
 * /proc rather than shelling out, so the containment cap below has no
 * external dependency. */
static long count_processes_for_uid(uid_t uid)
{
    DIR *proc = opendir("/proc");
    if (!proc)
        return -1;

```

```

long count = 0;
struct dirent *entry;
while ((entry = readdir(proc)) != NULL)
{
    if (entry->d_name[0] < '0' || entry->d_name[0] > '9')
        continue;

    char path[280];
    snprintf(path, sizeof(path), "/proc/%s", entry->d_name);

    struct stat st;
    if (stat(path, &st) == 0 && st.st_uid == uid)
        count++;
}
closedir(proc);
return count;
}

/* ----- monitor threads ----- */

void *timeout_monitor(void *arg)
{
    int timeout = *(int *)arg;

    struct timespec t0;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    while (atomic_load(&child_running))
    {
        struct timespec now;
        clock_gettime(CLOCK_MONOTONIC, &now);
        double elapsed = (now.tv_sec - t0.tv_sec) + (now.tv_nsec - t0.tv_nsec) / 1e9;

        if (elapsed >= timeout)
        {
            if (atomic_load(&child_running))
            {
                printf("[Monitor] Time limit exceeded.\n");
                /* Negative pid == signal the whole process group, so any
                 * processes the child forked (e.g. a fork-bomb payload)
                 * are terminated along with it, not left as orphans. */
                kill(-child_pid, SIGKILL);
                printf("[Monitor] Child terminated using SIGKILL.\n");
            }
            break;
        }

        usleep(100000); /* poll every 100ms instead of one long sleep,
                        so we notice promptly once the child exits */
    }

    pthread_exit(NULL);
}

void *resource_monitor(void *arg)
{
    (void)arg;

    while (atomic_load(&child_running))
    {
        double cpu = read_cpu_seconds(child_pid);
        long rss = read_rss_kb(child_pid);

        if (cpu >= 0.0)

```

```

    {
        if (cpu > peak_cpu_seconds)
            peak_cpu_seconds = cpu;
        printf("[Monitor] Sample: cpu=%.2fs mem=%ldKB\n", cpu, rss >= 0 ? rss : 0);
    }
    if (rss > peak_rss_kb)
        peak_rss_kb = rss;

    usleep(200000); /* 200ms sampling interval */
}

pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t timeout_thread, resource_thread;
    int timeout = 5;
    const char *target = (argc > 1) ? argv[1] : "./test_binary";

    clk_tck = sysconf(_SC_CLK_TCK);
    page_size_kb = sysconf(_SC_PAGESIZE) / 1024;

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (child_pid == 0)
    {
        printf("[Child] Executing %s...\n", target);

        /* Put the child in its own process group so the sandbox can
         * terminate the whole group (child + any processes it forks,
         * e.g. a fork-bomb payload) with a single kill(-pgid, ...)
         * instead of only the immediate child, which would otherwise
         * leave descendants running as orphans after the demo ends. */
        if (setpgid(0, 0) != 0)
            perror("[Child] setpgid");

        /* Containment: cap the number of processes this UID may hold,
         * relative to what's already running, so a fork-bomb style
         * payload cannot exhaust system-wide PIDs. Once the cap is
         * hit, fork() inside the child just fails with EAGAIN instead
         * of spawning further processes. */
        long current = count_processes_for_uid(getuid());
        struct rlimit rl;
        if (current > 0 && getrlimit(RLIMIT_NPROC, &rl) == 0)
        {
            rlim_t cap = (rlim_t)current + 40;

            if (rl.rlim_max != RLIM_INFINITY && cap > rl.rlim_max)
                cap = rl.rlim_max;

            rl.rlim_cur = cap;
            if (setrlimit(RLIMIT_NPROC, &rl) == 0)
                printf("[Child] Process limit capped at %ld.\n", (long)cap);
            else
                perror("[Child] setrlimit(RLIMIT_NPROC)");
        }
    }
}

```

```

    }

    char *args[] = {(char *)target, NULL};

    execve(target, args, NULL);

    perror("execve");
    exit(EXIT_FAILURE);
}

printf("[Parent] Target binary = %s\n", target);
printf("[Parent] Child PID = %d\n", child_pid);

pthread_create(&timeout_thread, NULL, timeout_monitor, &timeout);
pthread_create(&resource_thread, NULL, resource_monitor, NULL);

int status;

waitpid(child_pid, &status, 0);

atomic_store(&child_running, 0);

clock_gettime(CLOCK_MONOTONIC, &end);
double total_runtime = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

pthread_join(timeout_thread, NULL);
pthread_join(resource_thread, NULL);

const char *verdict = "UNKNOWN";
int signal_num = 0;
int exit_code = 0;

if (WIFEXITED(status))
{
    verdict = "EXITED";
    exit_code = WEXITSTATUS(status);
    printf("[Parent] Child exited normally.\n");
}
else if (WIFSIGNALED(status))
{
    verdict = "KILLED";
    signal_num = WTERMSIG(status);
    printf("[Parent] Child killed by signal %d\n", signal_num);
}

printf("[Parent] Total runtime: %.2fs\n", total_runtime);
printf("[Parent] Peak CPU time: %.2fs\n", peak_cpu_seconds);
printf("[Parent] Peak memory (RSS): %ldKB\n", peak_rss_kb);

FILE *log = fopen("sandbox.log", "a");

if (log)
{
    time_t now = time(NULL);

    fprintf(log, "---- Execution finished at %s", ctime(&now));
    fprintf(log, "Target binary : %s\n", target);
    fprintf(log, "Child PID      : %d\n", child_pid);
    fprintf(log, "Verdict        : %s", verdict);
    if (signal_num)
        fprintf(log, " (signal %d / %s)", signal_num, strsignal(signal_num));
    else
        fprintf(log, " (exit code %d)", exit_code);
    fprintf(log, "\n");
}

```

```

        fprintf(log, "Total runtime : %.2fs\n", total_runtime);
        fprintf(log, "Peak CPU time : %.2fs\n", peak_cpu_seconds);
        fprintf(log, "Peak memory   : %ldKB\n\n", peak_rss_kb);

        fclose(log);
    }

    printf("[Parent] Log written to sandbox.log\n");

    return 0;
}

```

B.2 benign.c

```

#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("[benign] starting, will exit after 1 second\n");
    sleep(1);
    printf("[benign] exiting normally\n");
    return 0;
}

```

B.3 cpu_hog.c

```

#include <stdio.h>

int main(void)
{
    printf("[cpu_hog] starting infinite CPU-bound loop\n");

    volatile unsigned long i = 0;
    while (1)
    {
        i++;
    }

    return 0;
}

```

B.4 fork_bomb.c

```

#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("[fork_bomb] starting fork loop (pid=%d)\n", getpid());

    while (1)
    {
        fork();
    }

    return 0;
}

```

B.5 ignore_sigterm.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

static void handle_sigterm(int sig)
{
    (void)sig;
    printf("[ignore_sigterm] received SIGTERM, ignoring it\n");
}

int main(void)
{
    signal(SIGTERM, handle_sigterm);
    printf("[ignore_sigterm] starting, will ignore SIGTERM and loop forever\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}
```

B.6 Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -O2
PTHREAD = -pthread

TEST_BINARIES = test_binary benign cpu_hog fork_bomb ignore_sigterm

all: sandbox $(TEST_BINARIES)

sandbox: sandbox.c
    $(CC) $(CFLAGS) $(PTHREAD) -o $@ $^

test_binary: test_binary.c
    $(CC) $(CFLAGS) -o $@ $^

benign: benign.c
    $(CC) $(CFLAGS) -o $@ $^

cpu_hog: cpu_hog.c
    $(CC) $(CFLAGS) -o $@ $^

fork_bomb: fork_bomb.c
    $(CC) $(CFLAGS) -o $@ $^

ignore_sigterm: ignore_sigterm.c
    $(CC) $(CFLAGS) -o $@ $^

clean:
    rm -f sandbox $(TEST_BINARIES) sandbox.log

.PHONY: all clean
```

Appendix C: Interconnection client

C.1 malicious_client.c

```
/*
 * malicious_client.c - the binary that connects Task 1 and Task 2.
 *
 * This is a HOSTILE program. It does two things:
 * Phase 1: it attacks the Task 1 authentication server. It connects to the
 *          same UNIX socket the real frontend uses (/tmp/auth_socket, from
 *          common.h) and tries a list of guessed passwords for "admin".
 * Phase 2: it goes rogue - an infinite CPU loop that never stops on its own,
 *          the kind of runaway process a sandbox is meant to contain.
 *
 * Run it BARE and it attacks the login server and then hangs the machine.
 * Run it UNDER the Task 2 sandbox and the sandbox kills it (SIGKILL) and logs
 * the containment. Same binary, two outcomes - that is the interconnection.
 *
 * Build (inside ~/Task1, because it needs common.h):
 * gcc -Wall -Wextra malicious_client.c -o test_binary
 * then hand "test_binary" to your Task 2 sandbox exactly like before.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

#include "common.h" /* reuse Task 1's protocol: LoginRequest, SOCKET_PATH, sizes */

/* Passwords an attacker would try against "admin". "Password123" is in the
 * list on purpose: if your backend is running, one guess will actually crack
 * it, which is the point the report makes about weak, hard-coded credentials. */
static const char *guesses[] = {
    "123456", "admin", "password", "root",
    "letmein", "Password123", "qwerty", "toor"
};

int main(void)
{
    printf("[malware] started, pid = %d\n", getpid());
    printf("[malware] Phase 1: attacking Task 1 auth server at %s\n", SOCKET_PATH);
    fflush(stdout);

    int total = sizeof(guesses) / sizeof(guesses[0]);

    for (int i = 0; i < total; i++) {
        int fd = socket(AF_UNIX, SOCK_STREAM, 0);
        if (fd < 0) {
            perror("[malware] socket");
            break;
        }

        struct sockaddr_un addr;
        memset(&addr, 0, sizeof(addr));
        addr.sun_family = AF_UNIX;
        strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);

        if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
            printf("[malware] attempt %d: auth server not reachable, "

```

```

        "moving on\n", i + 1);
        close(fd);

        break; /* backend serves one client then exits - stop probing */
    }

    LoginRequest req;
    memset(&req, 0, sizeof(req));
    strncpy(req.username, "admin", USERNAME_SIZE - 1);
    strncpy(req.password, guesses[i], PASSWORD_SIZE - 1);
    write(fd, &req, sizeof(req));

    char response[RESPONSE_SIZE];
    memset(response, 0, sizeof(response));
    read(fd, response, sizeof(response));
    printf("[malware] guess %d: admin / %-12s -> %s\n",
           i + 1, guesses[i], response);
    fflush(stdout);
    close(fd);

    if (strcmp(response, "SUCCESS") == 0) {
        printf("[malware] *** CREDENTIALS CRACKED: admin / %s ***\n",
               guesses[i]);
        break;
    }
}

printf("[malware] Phase 2: going rogue - infinite CPU abuse.\n");
printf("[malware] I will never stop on my own. Contain me if you can.\n");
fflush(stdout);

/* Runaway payload: burn CPU forever. The Task 2 sandbox's time limit and
 * SIGKILL are what stop this. Run bare, it never returns. */
unsigned long x = 0;
while (1) {
    x++;
    if ((x % 500000000UL) == 0) {
        printf("[malware] still running, still uncontained... (%lu)\n", x);
        fflush(stdout);
    }
}

return 0; /* never reached */
}

```

Appendix D: Sandbox log (raw output)

D.1 sandbox.log

```

---- Execution finished at Thu Jul 30 06:47:43 2026
Target binary : ./benign
Child PID     : 97227
Verdict       : EXITED (exit code 0)
Total runtime : 1.00s
Peak CPU time : 0.00s
Peak memory   : 1408KB

---- Execution finished at Thu Jul 30 06:47:49 2026
Target binary : ./cpu_hog
Child PID     : 97239
Verdict       : KILLED (signal 9 / Killed)

```

```
Total runtime : 5.06s  
Peak CPU time : 5.00s  
Peak memory   : 1236KB
```

```
---- Execution finished at Thu Jul 30 06:47:54 2026  
Target binary : ./fork_bomb  
Child PID     : 97291  
Verdict       : KILLED (signal 9 / Killed)  
Total runtime : 5.08s  
Peak CPU time : 4.82s  
Peak memory   : 1568KB
```

```
---- Execution finished at Thu Jul 30 06:47:59 2026  
Target binary : ./ignore_sigterm  
Child PID     : 97336  
Verdict       : KILLED (signal 9 / Killed)  
Total runtime : 5.09s  
Peak CPU time : 0.00s  
Peak memory   : 1380KB
```